

JQUERY

| 1. What is jQuery?

jQuery is a powerful and widely-used JavaScript library designed to simplify HTML document traversing, event handling, animating, and Ajax interactions for rapid web development. Released in 2006 by John Resig, jQuery has become a cornerstone of modern web development due to its ease of use and cross-browser compatibility.

At its core, jQuery allows developers to write less code while accomplishing more than they would with plain JavaScript. The library abstracts common JavaScript tasks into simple, concise methods that can be easily chained together for complex actions. For instance, instead of writing multiple lines of JavaScript to manipulate DOM elements, a developer can use a single jQuery call.

The syntax of jQuery is intuitive and leverages the CSS selector syntax, making it accessible for those familiar with web design. This means that selecting elements on a page is as straightforward as using CSS selectors, which streamlines the process of DOM manipulation.

Moreover, jQuery provides robust support for event handling, allowing developers to attach event listeners to elements with minimal code. This is particularly useful for creating interactive user interfaces where user actions trigger visual changes or data updates without requiring full page reloads.

In addition to its core functionalities, jQuery also offers plugins that extend its capabilities, allowing developers to integrate features such as image sliders, form validation, and more without having to build these functionalities from scratch.

Overall, jQuery has played a significant role in shaping the way interactive web applications are developed and remains relevant despite the emergence of modern frameworks like React, Angular, and Vue.js, due to its simplicity and effectiveness.

| Requirements for Using jQuery

- **Basic Knowledge of HTML, CSS, and JavaScript:** Understanding these languages is essential since jQuery is a JavaScript library.
- **Web Browser:** Any modern web browser (Chrome, Firefox, Safari, etc.) to test your applications.

- Text Editor or IDE: You can use any text editor (like Visual Studio Code, Sublime Text, or Notepad++) for writing your code.

| Installing jQuery

Installing jQuery in a web project can be accomplished in several straightforward ways, each catering to different project needs and preferences. The two most common methods are using a Content Delivery Network (CDN) and downloading the library directly.

- Using a CDN:

The easiest way to include jQuery in your project is by linking to a CDN. A CDN hosts jQuery files on servers distributed across the globe, allowing for faster load times and reduced bandwidth usage on your own server. To install jQuery via a CDN, you simply need to add a `<script>` tag in the `<head>` section of your HTML file. For example:

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

This line fetches the minified version of jQuery 3.6.0 from the official jQuery CDN. Using a CDN also has the added benefit of caching; if users have visited another site that uses the same version of jQuery from the same CDN, it will load faster because it may already be cached in their browser.

- Downloading jQuery:

Alternatively, you can download jQuery directly from the official jQuery website. This method is useful if you want to host the library on your own server or if you need to work offline. After downloading:

- Save the file in your project directory (e.g., `js/` folder).
- Link it in your HTML file like so:

```
<script src="path/to/your/jquery.min.js"></script>
```

This approach gives you complete control over the version of jQuery being used and allows you to modify it if necessary (though this is not commonly recommended).

Regardless of the installation method chosen, it is crucial to ensure that jQuery is included before any scripts that depend on it. This can typically be done by placing your custom scripts after the jQuery script tag in your HTML file.

Basic Syntax of jQuery

The basic syntax of jQuery is designed to be simple and intuitive, enabling developers to quickly select elements and perform actions on them with minimal code. At its core, the syntax follows this structure:

```
$(selector).action();
```

Here's a breakdown of each component:

- `$`: This symbol represents the jQuery function itself. It is often referred to as the "jQuery wrapper" and is used to access all jQuery functionalities.
- `selector`: This part specifies which HTML elements you want to manipulate. The selector can be any valid CSS selector (like an ID, class, or element type). For example:
 - `$("#myId")` selects an element with the ID `myId`.
 - `(".myClass")` selects all elements with the class `myClass`.
 - `("div")` selects all `<div>` elements.
- `action()`: After selecting an element(s), you can apply various actions or methods provided by jQuery. These methods can include manipulating styles, attributes, content, or handling events. For example:
 - `.hide()` hides the selected elements.
 - `.show()` displays them.
 - `.css("color", "red")` changes the text color of selected elements to red.

A practical example of this syntax would be:

```
$("p").css("color", "blue").fadeIn();
```

This line selects all `<p>` elements on the page, changes their text color to blue, and then fades them in smoothly.

Chaining methods is one of the powerful features of jQuery; you can apply multiple actions in a single statement by chaining them together using dot notation. This feature not only makes your code more concise but also enhances readability.

By understanding this basic syntax, developers can leverage jQuery's extensive library of functions to create dynamic and interactive web applications efficiently.

4. Example of using jQuery

To do list project

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Jquery assignment</title>

  <script src="jquery-3.6.0.js"></script>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <body>

    <div class="container">

      <h1>To-Do List</h1>

      <input type="text" id="taskInput" placeholder="Add a new task..."
required>

      <button id="addTaskButton">Add Task</button>

      <ul id="taskList"></ul>

    </div>

    <script>

      $(document).ready(function() {

        $('#addTaskButton').on('click', function() {
```

```
const taskText = $('#taskInput').val();

if (taskText) {

  // Create a new list item

  const listItem = $('<li></li>').text(taskText);

  // Create a delete button

  const deleteButton = $('<button class="delete-button">Delete</button>');

  listItem.append(deleteButton);


  // Append the new task to the list

  $('#taskList').append(listItem);


  // Clear the input field

  $('#taskInput').val('');


  // Mark task as completed when clicked

  listItem.on('click', function() {

    $(this).toggleClass('completed');

  });

  // Remove task when delete button is clicked

  deleteButton.on('click', function(event) {

    event.stopPropagation(); // Prevent triggering the click event on the list
    item
```

```
        $(this).parent().remove(); // Remove the parent list item
    });

}

});

// Allow pressing "Enter" key to add a task
$('#taskInput').on('keypress', function(event) {
    if (event.which === 13) { // Enter key
        $('#addTaskButton').click();
    }
});

});

</script>

</body>

</html>
```

Style Sheet

```
body {
    font-family: Arial, sans-serif;
    background-color: white;
    margin: 0;
    padding: 20px;
}
```

```
.container {  
    max-width: 100%;  
    margin: auto;  
    background: rgb(84, 206, 135);  
    padding: 20px;  
    border-radius: 5px;  
    box-shadow: 0 0 10px rgb(58, 70, 75);  
    margin-top: 50px;  
}  
  
h1 {  
    text-align: center;  
}  
  
input {  
    padding: 10px;  
    width: calc(100% - 22px);  
    margin-right: 10px;  
}  
  
button {  
    padding: 10px 15px;  
    color: blue;  
}
```

```
ul {  
    list-style-type: none;  
    padding: 0;  
}  
  
li {  
    padding: 10px;  
    border-bottom: 1px solid #ccc;  
    display: flex;  
    justify-content: space-between;  
    align-items: center;  
}  
  
.completed {  
    text-decoration: line-through;  
    color: gray;  
}  
  
.delete-button {  
    background-color: red;  
    color: white;  
    border: none;  
    padding: 5px 10px;  
    cursor: pointer;  
}
```

To illustrate how jQuery can be used in a practical scenario, let's consider a simple web application that allows users to toggle the visibility of a paragraph using buttons.

| **Project Overview**

➤ Purpose

The To-Do List application is designed to help users manage their tasks efficiently. Users can add, complete, and delete tasks, providing a simple interface for task management.

➤ Features

1. Add Tasks: Users can input a task in a text box and add it to the list by clicking an "Add Task" button or pressing the Enter key.
2. Display Tasks: Each task is displayed as a list item, making it easy to see all tasks at a glance.
3. Mark as Completed: Users can click on a task to mark it as completed, which visually indicates the task is done (typically through strikethrough styling).
4. Delete Tasks: Each task has an associated "Delete" button, allowing users to remove tasks they no longer need.

➤ Technologies Used

- HTML: For the structure of the application, including input fields, buttons, and lists.
- CSS: For styling the application, including layout and visual effects (e.g., strikethrough for completed tasks).
- JavaScript/jQuery: For interactivity, allowing users to add, complete, and delete tasks dynamically without refreshing the page.

➤ User Interface

The user interface typically includes:

- An input field for entering new tasks.
- An "Add Task" button to submit the new task.
- An unordered list () that displays all added tasks.
- Each task has a "Delete" button next to it for easy removal.

Workflow

➤ User Interaction

- The user types a task into the input field.
- The user clicks the "Add Task" button or presses Enter to add the task.

➤ Task Management

- The task appears in the list with a delete button.
- Clicking on a task marks it as completed (visually indicating it's done).
- Clicking the delete button removes the task from the list.

➤ Benefits

- **Simplicity:** The application is straightforward and easy to use, making it accessible for users of all technical levels.
- **Real-Time Updates:** Tasks are added and removed in real-time without needing to refresh the page, enhancing user experience.
- **Visual Feedback:** The ability to mark tasks as completed provides immediate visual feedback on progress.